IndustrialMind

A security layer that sits between a SCADA control station and a water-treatment PLC, and stops dangerous commands before they reach the plant.

What this document is. A complete, plain-language account of what we built, what we measured, and what we deliberately chose *not* to ship. Every figure in it was read automatically from the project's own result files when this PDF was generated — none of them is typed by hand — so each one can be reproduced by re-running the command named beside it.

849	4600/4600	0.50 ms	0.836
automated checks passing, 0 failing	rule-engine decisions correct	median decision time per command	ROC-AUC on real held-out SWaT data

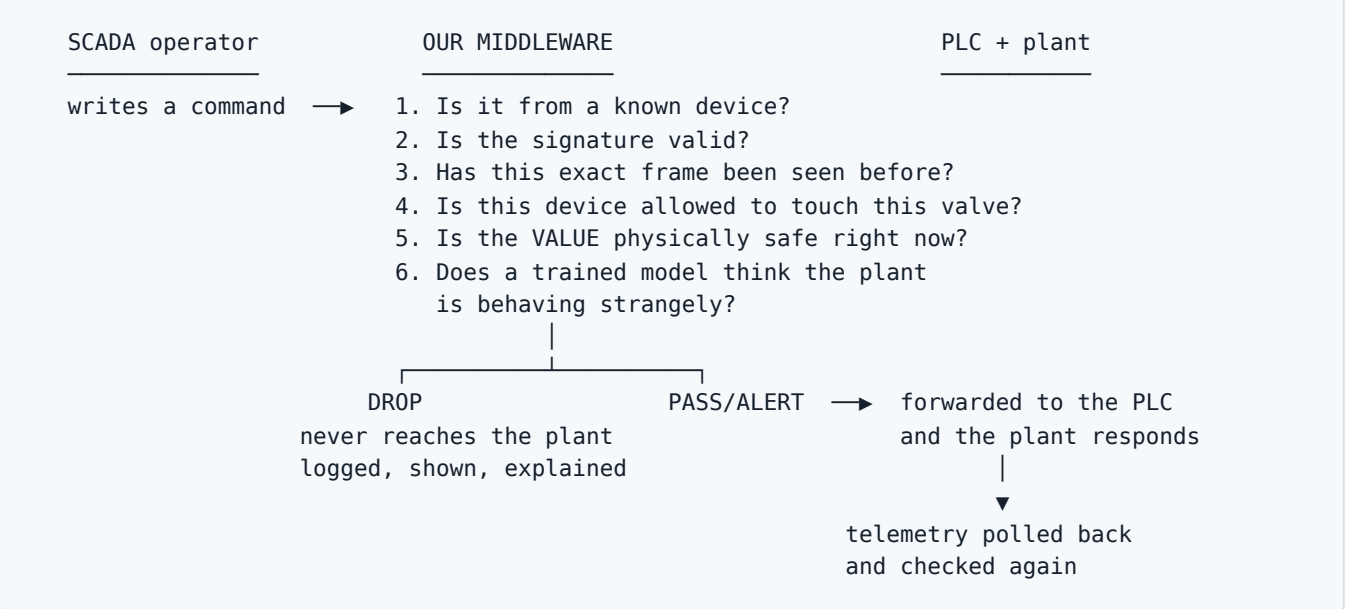
Built 21 October 2026 · git commit `371ce53` (69 commits) · 22,966 lines of Python
Python 3.13.16 · scikit-learn 1.9.1 · numpy 2.5.4
Generated by `python -m docs.guide_review.build_report`

1. The problem, in one page

A water-treatment plant is run by a **PLC** (a small industrial computer that opens valves and starts pumps) taking orders from a **SCADA** station (the operator's screen). They talk over **Modbus/TCP**, a protocol designed in 1979 for trusted serial cables. Modbus has **no passwords, no signatures and no encryption**. Any device that can reach the PLC's network port can tell it to do anything, and the PLC will obey.

That is not a theoretical weakness. It is how Stuxnet, the 2015/2016 Ukrainian grid attacks and the 2021 Oldsmar water incident all worked: the attacker did not break the protocol, because there was nothing to break.

Our approach. Rather than replacing the protocol — which no plant can afford to do — we put a security layer *in the wire*. Every command from SCADA now arrives at our middleware first. The middleware decides, in under a millisecond, whether to let the command through, and only then forwards it to the PLC. Nothing else changes: the SCADA software and the PLC are untouched and do not know we are there.



The two halves of that decision are deliberately different in kind:

- **The cyber half asks "who sent this, and is the message genuine?"** — authentication, replay protection, access control. These are **deterministic rules**. A signature is either valid or it is not; we never want a model guessing about that.
- **The physical half asks "would this command hurt the plant?"** — bounds, rate of change, and relationships between stages (for example, do not run a pump into a closed valve). Here we also run a **machine-learning model** trained on real plant data, because "the plant is behaving oddly" is a judgement no fixed rule captures well.

> **The key design decision.** The ML model is **advisory only**. It can raise an ALERT and colour the operator's screen, but it can **never block a command**. Blocking is done exclusively by rules a human can read and audit. This is deliberate: our own measurements (Section 5) show the model misses about half of all attacks and that about half its alarms are false. A component with those numbers must not be allowed to shut down a water plant.

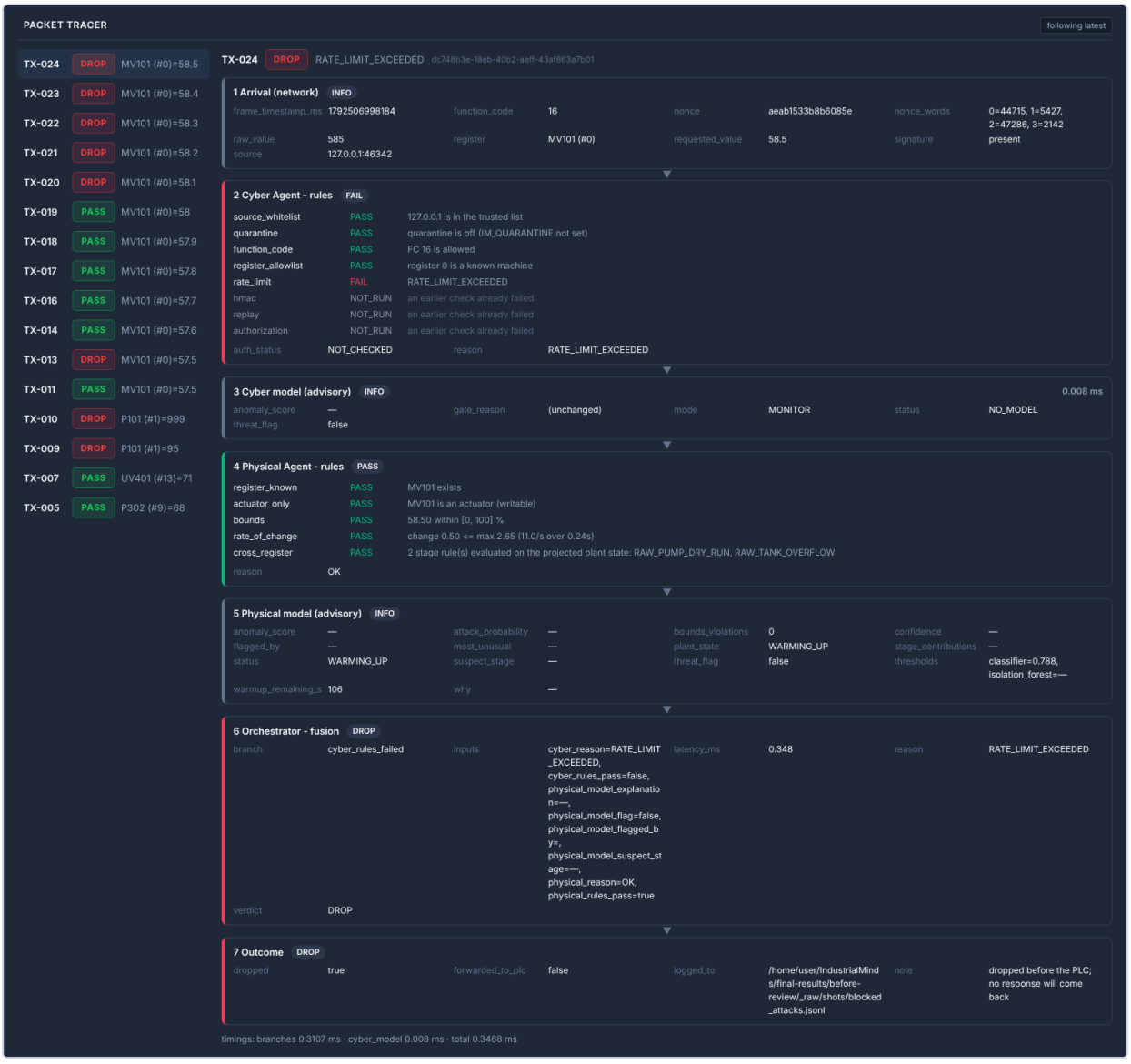
2. What we actually built

Part	What it does, in plain terms
PLC digital twin `plc/`	A simulated water-treatment plant: 5 stages, 20 measurement points, with real physics — tanks fill and drain, pumps affect pressure downstream, chemicals take time to act. It speaks real Modbus, so from the network it is indistinguishable from a real PLC. All its physical constants come from the real SWaT plant's measured data.
Middleware `middleware/`	The security layer itself. It pretends to be the PLC to SCADA, and pretends to be SCADA to the PLC, so it sees every command. Split into five focused modules: transport, decision, forwarding, polling and shared state.
Cyber agent	Eight deterministic checks in a fixed order: known source, valid function code, known register, rate limit, HMAC-SHA256 signature, replay/nonce reuse, quarantine, and an access-control policy.
Physical agent	Five deterministic checks: is this a real register, is it writable, is the value within measured safe bounds, is it changing too fast, and does it violate a cross-stage safety rule.
Physical ML model `models/`	Trained on the real SWaT dataset. Looks at a 120-second rolling window of all 20 sensors, computes 117 features, and estimates how likely it is that the plant is under attack. Advisory only.
Orchestrator	Combines the two halves into one verdict: any rule failure → **DROP**; all rules pass but the model is suspicious → **ALERT** (forwarded, but flagged); otherwise → **PASS**.
Dashboard `dashboard/`	A live operator screen: every decision, the plant's state, a per-hop "packet tracer" showing exactly which check stopped a command, correlated incidents, and the model's own explanation of why it flagged something.
Attacker `attacker/`	Ten scripted attacks plus a randomised mode, so the defence is tested against something that is not simply the mirror image of itself.

Making the system explain itself

A security tool that says "blocked" and nothing else is useless to an operator at 3 a.m. Three features exist purely so a human can understand what happened:

- **Packet tracer** — for every command, the exact sequence of checks, which one failed, and which never ran because an earlier one already failed.
- **Model explanation** — when the ML model flags something, it names the plant *stage* most responsible, by substituting that stage's raw sensor readings with the last known-normal readings and re-scoring.
- **Incident correlation** — forty separate blocked commands from one source become one incident with a severity, not forty rows to scroll past.



The packet tracer: every check, in order, with the one that failed and the ones that never ran.

3. The data we used, and our honesty rule

We used exactly one external dataset: **SWaT** (Secure Water Treatment) from iTrust, Singapore University of Technology and Design — a real, physical water-treatment testbed that researchers attacked under controlled conditions. After de-duplication it gave us **946,719 rows** at one reading per second across 51 sensor and actuator tags, with **35 labelled attack periods**.

> **The rule we set ourselves, and kept.** Anything SWaT cannot supply is either derived from SWaT's own columns, or clearly labelled synthetic. We never present self-generated data as real-world performance. This is why the results in Section 5 are separated into three sections that are never mixed: **real SWaT data**, **constructed rule scenarios**, and **digital-twin measurements**.

Things we did carefully that are easy to get wrong

- **We split the data by calendar time, not randomly.** Randomly shuffling time-series rows lets the model see the future: a window from 14:05 in training and 14:06 in testing are nearly identical, and accuracy becomes fiction.
- **We evaluate per attack segment, not per row.** One single attack in SWaT lasts about ten hours and accounts for 66% of all attack rows. Scoring row-by-row would let a model look excellent by detecting that one event and nothing else.
- **We report that our test block is not perfectly blind.** It was used for exactly two model refits. A pristine test set would be better; saying so is better than pretending.
- **The feature-extraction code is literally shared between training and the live system** (`swat/`), so the model cannot be fed differently in the lab than in production — the most common cause of models that work on paper and fail in the field.

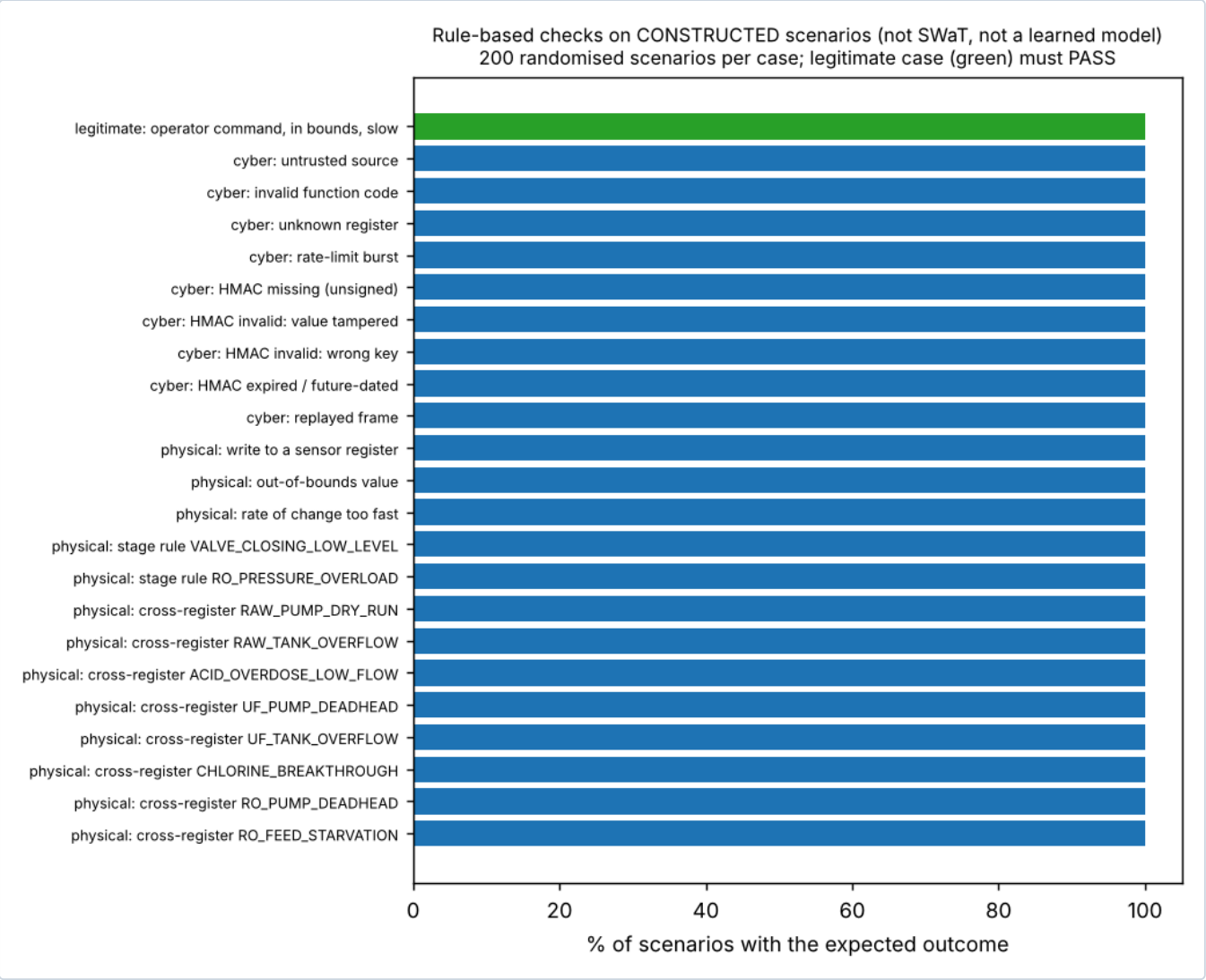
4. Results — part 1: the deterministic rules

These are the checks that are allowed to block. They are specifications, not predictions: an unsigned command *must* be rejected, every time. We test each case 200 times with randomised values.

4600/4600	100%	0
correct decisions	accuracy	legitimate commands wrongly blocked

Layer	Scenario	Tried	Correct	Rate
legitimate	operator command, in bounds, slow	200	200	**100%**
cyber	untrusted source	200	200	**100%**
cyber	invalid function code	200	200	**100%**
cyber	unknown register	200	200	**100%**
cyber	rate-limit burst	200	200	**100%**
cyber	HMAC missing (unsigned)	200	200	**100%**
cyber	HMAC invalid: value tampered	200	200	**100%**
cyber	HMAC invalid: wrong key	200	200	**100%**
cyber	HMAC expired / future-dated	200	200	**100%**
cyber	replayed frame	200	200	**100%**
physical	write to a sensor register	200	200	**100%**
physical	out-of-bounds value	200	200	**100%**
physical	rate of change too fast	200	200	**100%**
physical	stage rule VALVE_CLOSING_LOW_LEVEL	200	200	**100%**
physical	stage rule RO_PRESSURE_OVERLOAD	200	200	**100%**
physical	cross-register RAW_PUMP_DRY_RUN	200	200	**100%**
physical	cross-register RAW_TANK_OVERFLOW	200	200	**100%**
physical	cross-register ACID_OVERDOSE_LOW_FLOW	200	200	**100%**
physical	cross-register UF_PUMP_DEADHEAD	200	200	**100%**
physical	cross-register UF_TANK_OVERFLOW	200	200	**100%**
physical	cross-register CHLORINE_BREAKTHROUGH	200	200	**100%**
physical	cross-register RO_PUMP_DEADHEAD	200	200	**100%**
physical	cross-register RO_FEED_STARVATION	200	200	**100%**

> **How to read this honestly.** 100% here does *not* mean "we detect 100% of attacks". It means these rules do exactly what they are specified to do, with no false positives on legitimate traffic. A rule only catches the attack it was written for; the model in Section 5 exists precisely because rules cannot cover everything. Reproduce with `python -m benchmark.run_all`.



Every rule-based scenario and its outcome.

5. Results — part 2: the ML model on real SWaT data

This is the honest, uncomfortable section, and the one worth spending time on with a reviewer. The model was trained on real SWaT data and then run **once** on a held-out block of 28,080 time windows (37.5 hours of benign operation) that it had not been tuned on.

0.836	0.461	0.513	0.485
ROC-AUC	precision	recall	F1 score

8/10

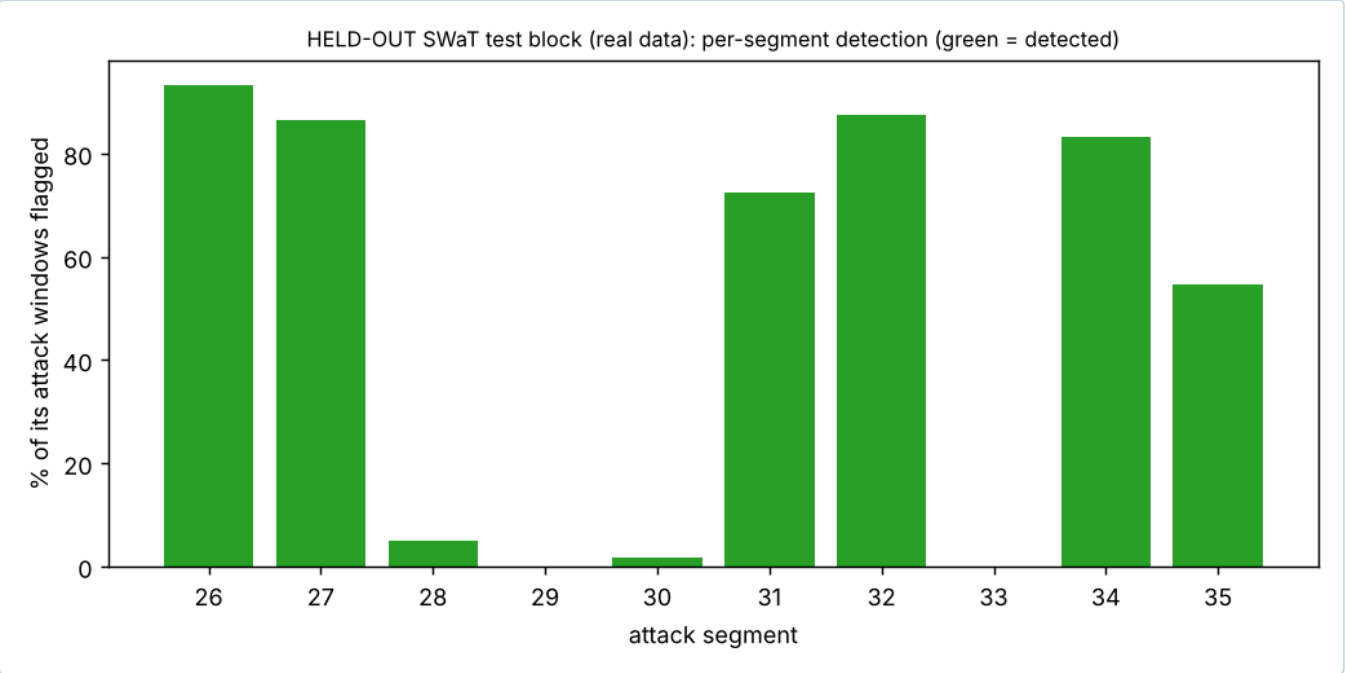
attacks detected

> **What these numbers mean, stated plainly.** Recall 0.513 means the model **misses roughly 49% of attack windows**. Precision 0.461 means **roughly 54% of its alarms are false**. Its raw accuracy is 95.89% — but a model that simply said "never an attack" would score 96.22% on this data, because attacks are rare. **Quoting the accuracy figure alone would be misleading, so we do not.** This single fact is the main reason the model is advisory and cannot block.

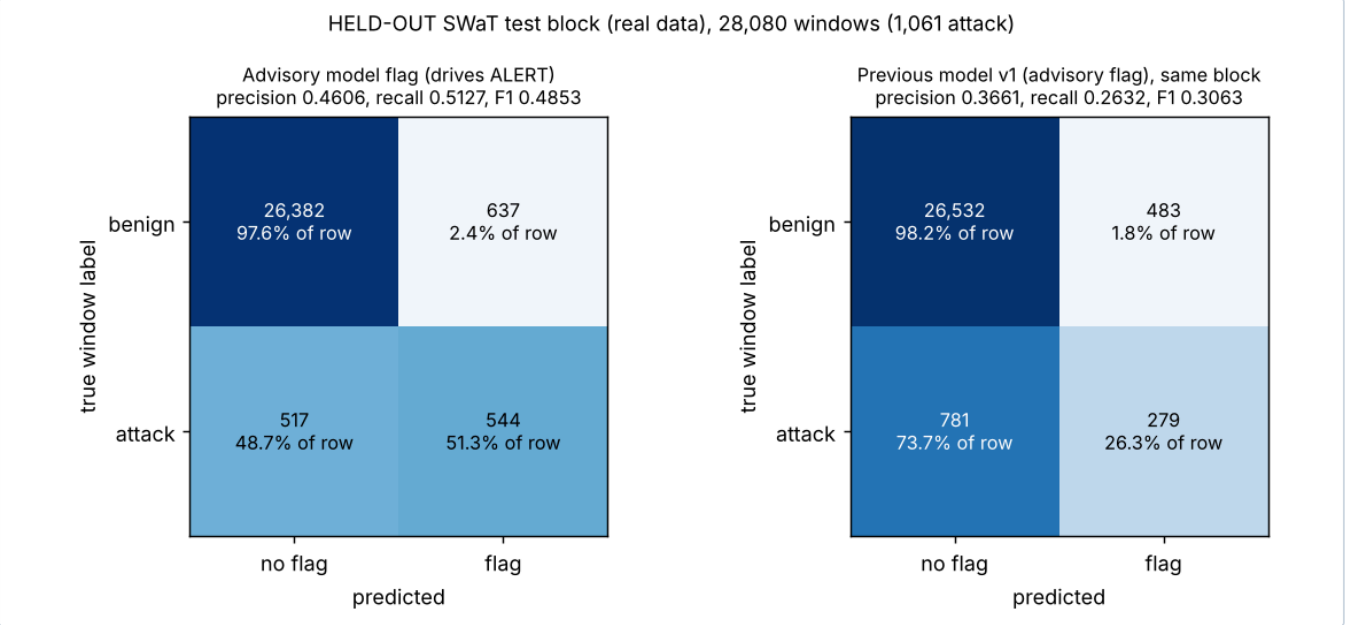
The more useful view is **per attack event**, because an operator cares whether an attack was caught at all, not how many individual seconds were flagged:

Attack segment	Duration	Windows flagged	Outcome	Time to detect
26	600 s	112/120	**detected**	40 s
27	445 s	77/89	**detected**	55 s
28	100 s	1/20	**detected**	60 s
29	480 s	0/96	**MISSED**	—
30	540 s	2/108	**detected**	75 s
31	470 s	68/94	**detected**	130 s
32	280 s	49/56	**detected**	35 s
33	400 s	0/80	**MISSED**	—
34	300 s	50/60	**detected**	50 s
35	1690 s	185/338	**detected**	85 s

8 of 10 attacks were detected, with a median detection time of **58 seconds**, at a cost of **1.17 false-alarm episodes per hour** of normal running. Put in operator terms: the model catches most real attacks within a minute, and interrupts the operator about once an hour when nothing is wrong. That trade-off is a deliberate setting we can tune (`IM_PHYSICAL_SENSITIVITY`) without retraining.



Per-attack-segment detection on the held-out SWaT test block.



Window-level confusion matrix, held-out SWaT test block.

6. Results — part 3: the whole system, running

The two sections above test components. This one starts the **real programs** — PLC, middleware, dashboard — on real network sockets, runs every attack the attacker tool knows plus labelled legitimate traffic, and reads the outcome back out of the dashboard's own API. Nothing is simulated or asserted; it is what the running system actually did.

66	1.00	0.58	0
commands sent	precision	recall	legitimate commands blocked

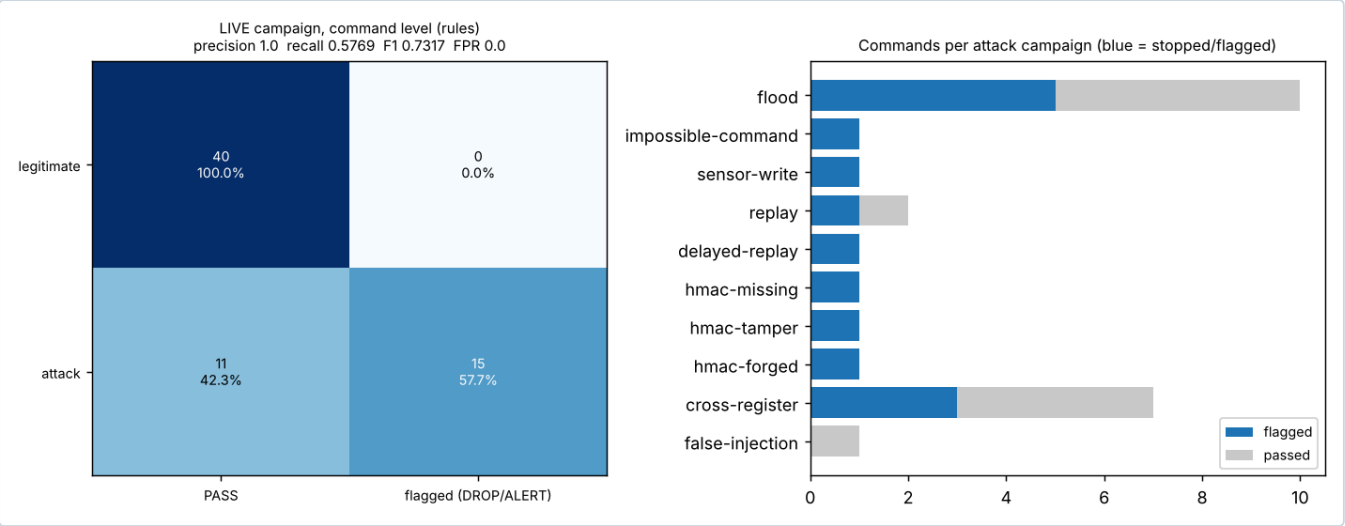
9/9

attack campaigns detected

Attack campaign	Type	Commands	Blocked	Allowed
legitimate operator traffic	benign	40	0	40
flood	attack	10	5	5
impossible-command	attack	1	1	0
sensor-write	attack	1	1	0
replay	attack	2	1	1
delayed-replay	attack	1	1	0
hmac-missing	attack	1	1	0
hmac-tamper	attack	1	1	0
hmac-forged	attack	1	1	0
cross-register	attack	7	3	4
false-injection	attack	1	0	1

> **Every attack campaign that passed through the middleware was detected (9/9), and not one legitimate command out of 40 was blocked.** Note that "Allowed" is not always a failure: in a flood, the first few commands are legitimate-looking and only the burst is an attack, so blocking 5 of 10 is the correct behaviour, not a miss.

The one campaign that is not detected inline is `false-injection`, and we report it deliberately. It does not go through the middleware at all — it opens its own socket straight to the PLC. No in-line device can block traffic that never reaches it. We now defend it in two ways: the PLC itself refuses writes from anything that is not the middleware (Section 8), and the physical model still sees the plant misbehaving afterwards. On a single-laptop demo every process shares the address `127.0.0.1`, so the PLC cannot tell the attacker from the middleware and the write is allowed; on the four-laptop setup, where the addresses differ, it is refused. We keep this row in the results instead of quietly removing it, because a results table that only contains wins is not evidence.



The live campaign: what the running system did with each attack.

7. Results — part 4: speed, and the digital twin

Is it fast enough to sit in the wire?

A security layer in the control path must not slow the plant down. Measured over 3,000 commands, repeated 3 times:

0.50 ms
median (p50)

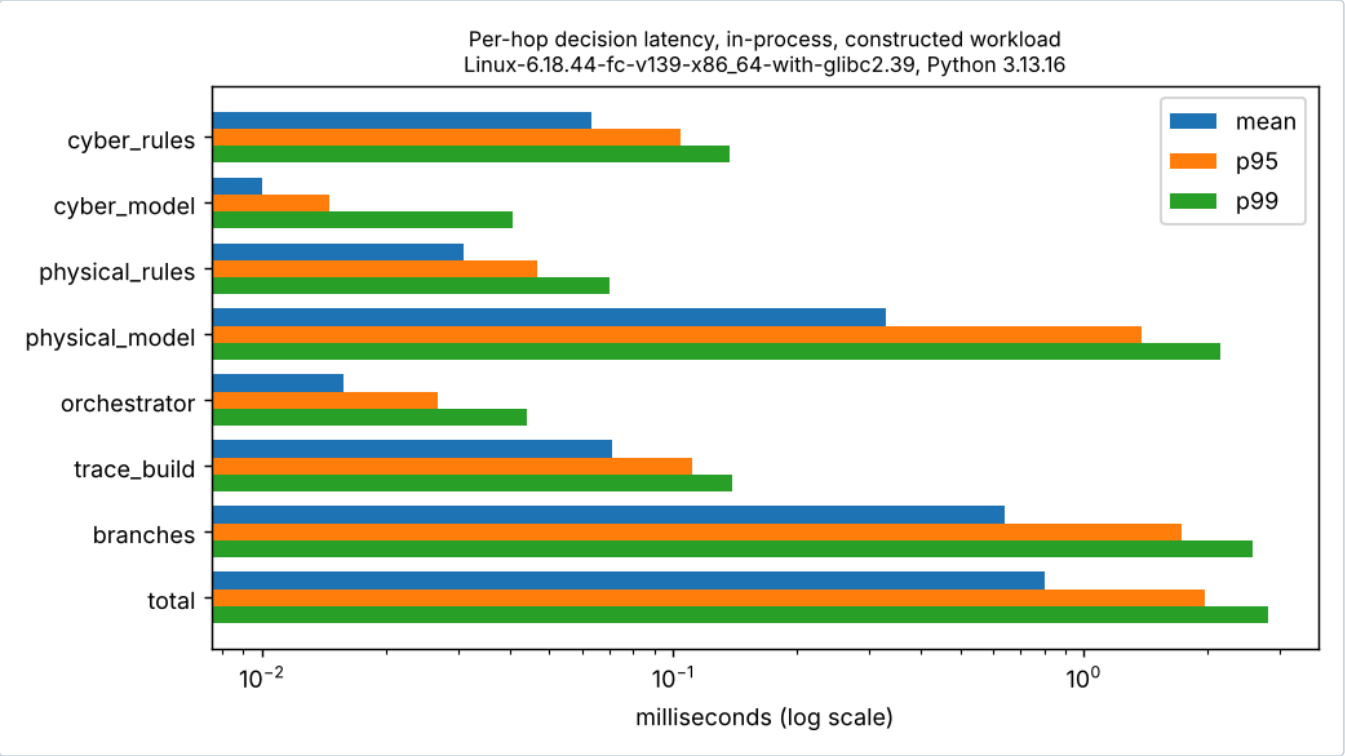
0.80 ms
mean

1.97 ms
95th percentile

2.80 ms
99th percentile

Stage of the decision	p50 (ms)	mean (ms)	p95 (ms)	p99 (ms)
cyber rules	0.059	0.064	0.104	0.138
cyber model	0.009	0.010	0.015	0.041
physical rules	0.028	0.031	0.047	0.070
physical model	0.009	0.330	1.377	2.145
orchestrator	0.014	0.016	0.027	0.044
trace build	0.067	0.071	0.111	0.139
branches	0.342	0.640	1.724	2.569
Total	**0.501**	**0.801**	**1.972**	**2.797**

For context, this plant's control loop runs at about one command per second. Even the 99th-percentile decision costs 2.80 ms — roughly **0.3% of the time available**. The security layer is free, in practice.



Where the decision time goes.

Does it catch attacks on a live plant?

We ran the model against our digital twin for 4 hours of healthy operation, then staged five realistic sabotage scenarios.

0

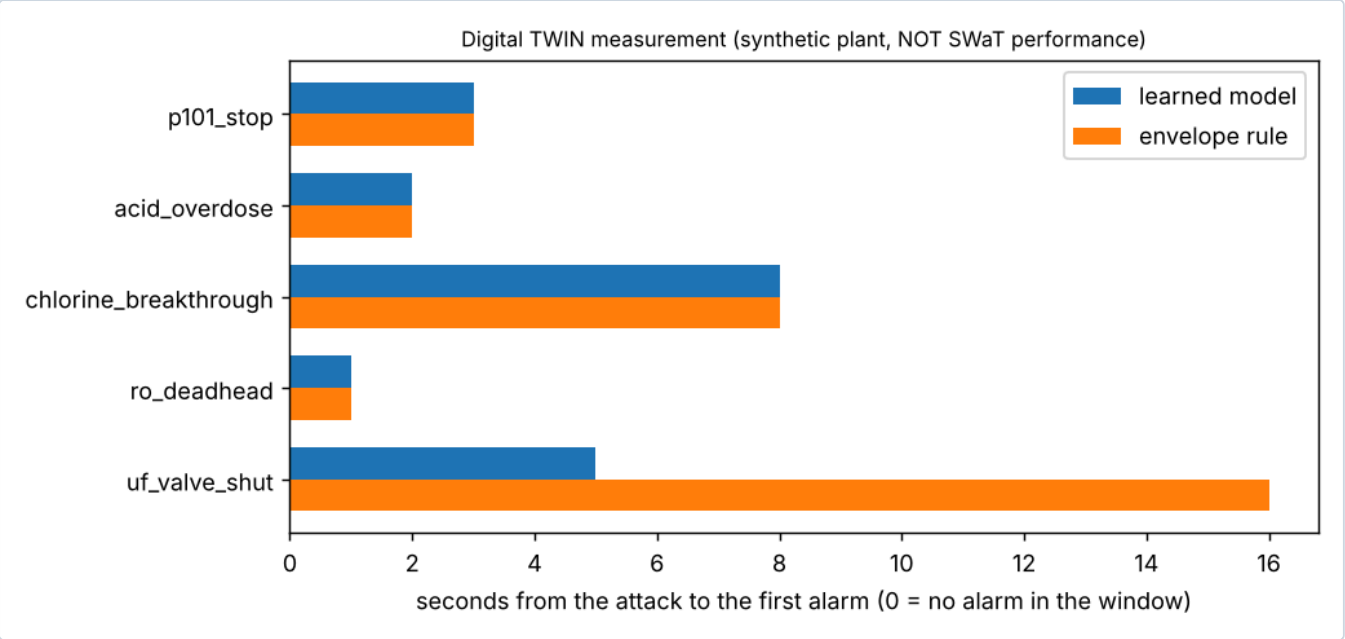
false alarms in 4 h of normal running

1–8 s

time to flag a sabotage

Sabotage scenario	False alarms before it started	Flagged after
p101_stop (raw pump stops)	0 s	**3 s**
acid_overdose (HCl max, raw pump low)	0 s	**2 s**
chlorine_breakthrough (NaOCl max, UV off)	0 s	**8 s**
ro_deadhead (RO pump max, reject valve shut)	0 s	**1 s**
uf_valve_shut (UF feed valve closes)	0 s	**5 s**

> **This is a twin measurement, not a SWaT result.** The twin is calibrated to SWaT's measured statistics, but it is still our own simulation, and a model tends to do better on a world that resembles its training data than on the real one. We report it as what it is: evidence that the model responds correctly and quickly to a physical attack in progress, not as a detection rate.



How quickly each staged sabotage was flagged on the twin.

8. Proving the system really does what we claim

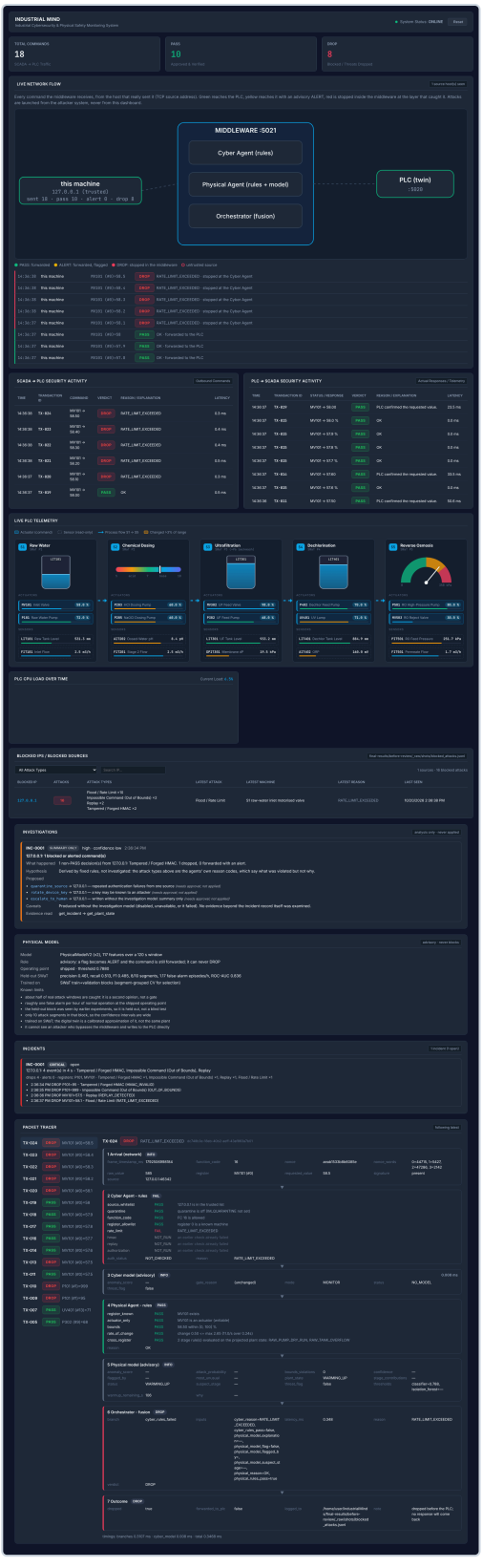
It is easy to write software that reports its own success. The strongest piece of evidence in this project is a test that checks our claims against **the network itself**.

We simulate the full four-machine lab on one computer, each role on its own address, and capture the real network traffic with **tshark** (the command-line Wireshark). Then we cross-check **four independent records** of the same events, which are produced by four different parts of the system and have no reason to agree unless the system is behaving as described:

1. the **captured network packets** — the ground truth, taken off the wire;
2. the middleware's **packet tracer**, matched to the packets by the signature's nonce;
3. the **blocked-attack log** written to disk;
4. the **dashboard's live feed**.

> **Result: 24 signed command frames captured, 24 consistent with the tracer, 0 mismatches.** The bypass write appears on the PLC leg only and never on the middleware leg — exactly as our claim predicts — and the PLC's own log shows it being refused. Reproduce with `python -m tools.sim_four_laptops`.

Because the capture is a standard `.pcapng` file, a reviewer can open it in Wireshark and see every signed frame, every forwarded command and the bypass attempt, independently of anything we wrote.



The live operator dashboard during a run (full page, scaled to fit).

9. What we tried and deliberately did not ship

Three machine-learning models were built, trained on real SWaT data, measured, and then **rejected**. We kept the code and the reports. A negative result that is measured and documented is a finding; quietly deleting it would make the project look better and be worth less.

What we tried	What we measured	Decision
ML model for the cyber half — learn to spot malicious command patterns	Test ROC-AUC 0.63, F1 0.137 — barely better than guessing	**Rejected.** The cyber branch is rules-only. SWaT contains no network-layer data, so there was nothing real to learn from; authentication should be deterministic anyway.
Learned fusion — a model to combine the cyber and physical verdicts, instead of fixed logic	Validation AUC 0.93 looked excellent, but test AUC was 0.68 versus 0.83 for the physical model alone, and it detected 4/10 attack segments instead of 6/10	**Rejected.** A textbook case of a model that fits the validation set. The orchestrator uses a readable four-branch rule table.
Physical model v3 — multi-scale features, stacked per-stage models	F1 0.352 against 0.432 for the simpler v2 baseline; 6/10 segments against 8/10	**Not promoted.** More complexity, worse results. v2 stays live; v3 is available behind a flag for anyone who wants to reproduce the comparison.

> There is a general lesson here worth saying out loud to a reviewer: in all three cases the more sophisticated option lost to the simpler one on honestly held-out data. The only reason we know that is that we measured it on a test block we had not tuned on.

10. The independent audit, and what it changed

Rather than finish and declare success, we ran a deliberately hostile review of our own system — written to find reasons it would fail in a real plant, with every finding required to cite a file and line. The state of the system before the review was frozen in `final-results/before-review/` and never edited, so the before-and-after is verifiable rather than remembered.

The review found a critical bug that all 849 of our passing tests had missed, because every test checked a component against its own specification and none checked what an operator actually needs. All findings have since been fixed and each is now pinned by a test written from the operator's point of view.

What was wrong	Why it mattered	Before → after
C1. A command approved by the middleware was reported to the operator as a green PASS even when the PLC was unreachable and never received it	The classic control-room failure: the screen and the plant disagree, and the operator believes the screen	reported **PASS** → reported **UNDELIVERED**
C2. An unresponsive PLC stalled the whole middleware, for every client at once	One flaky network cable would have frozen the entire demo and looked like our software had crashed	**2,002 ms** → 504 ms → **1.9 ms** per command
C3. A library wrote ~99 error lines into the log during a 30-second outage	Every blocked attack would be buried in library noise in the log the operator reads	**99 lines** → **1 line**
C4. The PLC poll retried a dead PLC five times a second, forever	~18,000 connection attempts an hour; on a real plant network an intrusion-detection system would flag our security device as the noisiest thing present	**18,000** → **720** attempts/hour
C5. Replay protection read and then wrote shared state with no lock	Safe only by accident. The very next threading change would have let two copies of the same captured command both be accepted	unsynchronised → **12 threads, 1 accepted**
S1. The PLC accepted writes from anything on the network	The entire product assumes the middleware is in the path — and nothing enforced that	bypass worked → **bypass refused**
S4. The auto-quarantine could be turned against the operator	An attacker forging the operator's address could get the *real* operator locked out — our defence becoming the attack	40 forged frames locked the operator out → **no effect**
A4. All forensic evidence lived only in the dashboard's memory	Restarting the screen destroyed the whole record of an incident	lost on restart → **survives restart**
N2. The dashboard loaded its chart library and icons from the internet	The lab is air-gapped, so the chart silently rendered as a blank box	blank box → **renders offline**

> **Why this section belongs in the report.** Finding and fixing a critical bug in your own work, with the before-state preserved so the claim can be checked, is stronger evidence of engineering maturity than a project that reports no problems at all. Every project has defects; most do not go looking for them.

11. Security hardening

Protection	What it stops
HMAC-SHA256 signed commands	Forged or tampered commands. Every command carries a signature over its register, value, a random number and a timestamp.
Nonce + replay window	An attacker recording a genuine command and sending it again later.
Per-device keys	A key stolen from one machine being used from another — each device is verified with its own key.
Access-control policy	A legitimate, authenticated device touching equipment it has no business touching.
Auto-quarantine	A device that repeatedly sends violations it genuinely signed is cut off for a while, with the hold doubling for repeat offences. Only authenticated violations count, so it cannot be turned against the operator.
PLC write allow-list	Anything that tries to skip the middleware and command the plant directly.
Fail-closed design	If any part of the decision pipeline throws an error, the command is DROPPED, never waved through.

12. Known limitations — stated plainly

1. **The model misses about half of attack windows and about half of its alarms are false.** It is advisory for exactly this reason. Improving it needs more and more varied real attack data than SWaT provides.
2. **The cyber rules cannot be validated against real-world data.** SWaT has no network traffic, no signatures and no source addresses. They are tested against specifications and constructed scenarios, and we label them as such rather than inventing a detection rate.
3. **The results are reproducible on one machine.** The SWaT archive is licensed and cannot be redistributed, so anyone re-running the model numbers must obtain it themselves.
4. **The four-laptop deployment has been verified on loopback, not yet on four physical laptops.** The capture, the cross-check and the tooling all work; the hardware run is outstanding.
5. **A spoofed source address still defeats address-based protections.** Network segmentation remains the real control; our allow-list is defence in depth, and we say so in the code.
6. **The dashboard runs on a development web server**, appropriate for a lab demonstration, not for deployment.

13. How to check any claim in this document

Claim	Command that regenerates it
All automated checks pass (849 checks)	`python -m tests.run_all`
Rule accuracy, latency, twin detection	`python -m benchmark.run_all`
Model numbers on real SWaT	`python -m benchmark.run_all --data-dir <SWaT archive>`
Live attack campaign against the running system	`python -m benchmark.evidence_pack`
Four-record cross-check and Wireshark capture	`python -m tools.sim_four_laptops`
The audit, its findings and the fixes	`final-results/AUDIT.md`
The frozen pre-audit state	`final-results/before-review/`
This document	`python -m docs.guide_review.build_report`

14. Summary

We built a working inline security layer for an industrial control protocol that has no security of its own, and measured it honestly.

- **It works.** 4600 of 4600 rule decisions correct; every attack campaign that passes through it is detected; no legitimate command was wrongly blocked.
- **It is fast enough to be invisible** — 0.50 ms median, against a one-second control loop.
- **It is honest about its model.** Trained on real data, measured on a held-out block, reported with its weaknesses, and kept advisory because of them.
- **It explains itself** — per-hop tracing, stage-level attribution, correlated incidents.
- **It has been checked against the network itself** — 24 frames, 24 consistent, 0 mismatches.
- **It was audited adversarially**, and the defects found were fixed and pinned by tests, with the before-state preserved.

The three rejected models are, in their way, the clearest evidence of the method: each was built, measured on data it had not been tuned on, found wanting, and left out of the shipped system with its numbers recorded.
